CHAPTER 4

Pattern Matching Your Way to Success

Pattern matching will change the way that you write code. It's a simple—yet extremely powerful—feature that's built into the foundation of Elixir. Pattern matching is used in function definitions, variable assignments, and control flows—it's a core port of the language's design. And once you master it, you won't want to go back.

In Ruby (and most languages), the core structures for control flow are if statements. This is easy to use if you want to check whether a value is one of two candidate values—true or false. It becomes cumbersome if you want to check whether a value is one of many possible values—based on string contents, array values, map keys, and so on. Elixir has if statements, but it also has something more powerful.

In this chapter, you'll see how Elixir's case statement completely replaces if and switch statements. Elixirists use case statements a lot, so it's good to get comfortable with it. Luckily, it's also simple. You just need to know how pattern matching works. The syntax of pattern matching is simple, but its roots run deep and it takes a little bit of practice to get used to. This chapter builds up slowly so that you have everything you need to be confident with patterns.

We'll start by looking at the most basic forms of pattern matching. Then, you'll see how case statements are used in control flow. Finally, we'll combine everything to see how pattern matching affects function definitions and makes recursive functions much easier to write.

Pattern matching is a game changer, so let's dive in!